

MODULE 3: Data Manipulation, Retrieval and Subqueries

Course: SWE 306 | Level 300 | Instructor: Miss Precious Oben

1. EER Diagrams in MySQL Workbench

MySQL Workbench uses the Enhanced Entity-Relationship (EER) diagram to represent database schemas visually. Two types of relationships are represented in physical EER diagrams: identifying and non-identifying relationships.

1.1 Identifying Relationships

An identifying relationship occurs when the primary key of the child entity includes the primary key of the parent entity as part of its own key. This signifies a strong dependency where the child entity cannot exist without the parent.

In MySQL Workbench, an identifying relationship is drawn as a solid line between tables.

Example: An Enrollment entity has a composite primary key made up of StudentID from the students table and CourseID from the courses table. An enrollment record cannot exist unless both a student and a course exist. The ordinality is therefore mandatory on both sides.

```
-- Identifying relationship: composite PK made up of both FKs
CREATE TABLE enrollments (
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enroll_date DATE DEFAULT (CURRENT_DATE),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

1.2 Non-Identifying Relationships

A non-identifying relationship occurs when the primary key of the child entity does not include the parent entity's primary key. The child can exist independently of the parent but still holds a reference to it. The foreign key in the child table is just a regular column, not part of the child's own primary key.

In MySQL Workbench, a non-identifying relationship is drawn as a dashed line between tables.

These relationships can be optional, where the foreign key can be NULL, or mandatory, where the foreign key must always have a value.

```
-- Non-identifying relationship: child has its own independent PK
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    room_id INT, -- nullable: not all students have a room yet
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

	Identifying	Non-Identifying
Child PK includes parent PK?	Yes	No
Child exists without parent?	No	Yes
Workbench line style	Solid	Dashed

Ordinality	Always mandatory	Can be optional or mandatory
Example	Enrollment needs both Student and Course	Student can exist without a room assigned

1.3 Physical ER Diagram Symbols

In a physical EER diagram, crow's foot notation is used to show the cardinality and ordinality of each side of a relationship. The symbols appear at the end of the relationship line closest to the table they describe.

Symbol	Meaning	Represents
Single bar	One	Exactly one instance must participate
Double bar	One and only one	Mandatory participation of exactly one
Crow's foot <<<	Many	Multiple instances can participate
Circle O	Zero	Participation is optional (can be zero)
O + bar O	Zero or one	Optional, but maximum of one
O + crow O<<<	Zero or many	Optional participation, no upper limit
+ crow <<<	One or many	At least one, no upper limit

Note: These symbols always come in pairs, one on each end of the relationship line. Read them from left to right: the symbol near table A describes how A participates from B's perspective, and the symbol near table B describes how B participates from A's perspective.

2. Benefits of Data Modelling

Data modelling is not just a design exercise. It produces direct, measurable benefits throughout the lifecycle of a database system. The table below summarises the key benefits and what each means in practice.

Benefit	What It Means in Practice
Improved Data Quality	Enforcing constraints and relationships at the model level reduces redundancy and prevents bad data from entering the system.
Facilitates Communication	A visual model gives both business stakeholders and technical teams a shared language for discussing the system, reducing misunderstandings.
Enhances Data Integration	When data comes from multiple sources, a model defines how entities from different systems relate, enabling cleaner integration and consolidation.
Simplifies Database Design	The model serves as a blueprint, allowing designers to spot problems and test changes visually before writing a single line of SQL.
Supports Business Rules	Business rules such as a transaction must have a sender or a room cannot exceed its capacity are baked into the model as constraints, making the database enforce them automatically.

Eases Maintenance and Scalability	A well-designed model is easier to extend. New tables and relationships can be added without breaking existing structures if the original design was clean.
Optimises Performance	A well-structured model with appropriate normalisation and indexing reduces redundancy and organises data so queries run efficiently.
Compliance and Security	Access control and data protection requirements can be designed into the model, defining who can see and manipulate which data.

3. Data Manipulation and Retrieval

Data manipulation in SQL is performed using DML (Data Manipulation Language) commands. These four commands cover every operation you will need to perform on data stored in a database.

Command	Purpose	Used With WHERE?	Example
INSERT	Add new rows to a table	No	INSERT INTO users (...) VALUES (...)
SELECT	Retrieve data from one or more tables	Yes	SELECT * FROM users WHERE city = 'Buea'
UPDATE	Modify existing values in a table	Yes (always)	UPDATE users SET balance = 500 WHERE user_id = 1
DELETE	Remove existing rows from a table	Yes (always)	DELETE FROM users WHERE is_verified = 0

Warning: *UPDATE and DELETE without a WHERE clause affects every single row in the table. Always run a SELECT with the same WHERE clause first to preview what will be affected before executing either command.*

Data retrieval becomes more powerful when tables are related through primary and foreign keys. To pull meaningful data from related tables at the same time, SQL provides the JOIN.

- **Primary Key:** A column or group of columns that uniquely identifies each row in a table.
- **Foreign Key:** A column or group of columns in one table that references the primary key of another table, creating the link between them.

4. SQL JOINS

A JOIN combines rows from two or more tables based on a related column between them. Without JOINS, you would have to run multiple separate queries and manually piece the results together. JOINS allow you to retrieve combined data from related tables in a single query.

The general syntax for all JOINS follows the same pattern:

```
SELECT column_name(s)
FROM table1
JOIN_TYPE table2 ON table1.common_column = table2.common_column;
```

4.1 INNER JOIN

INNER JOIN returns only the rows where there is a matching value in both tables. Rows from either table that have no match in the other table are excluded from the result entirely.

Use INNER JOIN when you only want to see rows that have a corresponding entry in both tables.

```
-- Show each transaction with the name of the sender
-- Only transactions with a matching user record are returned
SELECT t.reference, t.amount, t.status,
       CONCAT(u.first_name, ' ', u.last_name) AS sender_name
FROM transactions t
INNER JOIN users u ON t.sender_id = u.user_id;
```

4.2 LEFT JOIN (LEFT OUTER JOIN)

LEFT JOIN returns all rows from the left table and the matching rows from the right table. Where there is no match in the right table, the result shows NULL for all columns from the right table.

Use LEFT JOIN when you want every row from the left table to appear in the result, even if there is no corresponding row in the right table.

```
-- Show all users and their transactions
-- Users with no transactions still appear, with NULL for transaction
columns
SELECT u.first_name, u.last_name, t.transaction_id, t.amount
FROM users u
LEFT JOIN transactions t ON u.user_id = t.sender_id;
```

4.3 RIGHT JOIN (RIGHT OUTER JOIN)

RIGHT JOIN returns all rows from the right table and the matching rows from the left table. Where there is no match in the left table, the result shows NULL for all columns from the left table.

Use RIGHT JOIN when you want every row from the right table to appear, regardless of whether there is a match on the left. In practice, most developers rewrite RIGHT JOINS as LEFT JOINS by swapping the table positions, as this is easier to read.

```
-- Show all transactions even if the sender user record no longer exists
SELECT u.first_name, t.reference, t.amount
FROM users u
RIGHT JOIN transactions t ON u.user_id = t.sender_id;
```

4.4 FULL JOIN (FULL OUTER JOIN)

FULL JOIN returns all rows from both tables. Where there is no match on either side, NULL appears for the columns from the unmatched table. MySQL does not support FULL JOIN natively, but it can be emulated by combining a LEFT JOIN and a RIGHT JOIN with UNION.

```
-- Emulating FULL JOIN in MySQL using UNION
SELECT u.first_name, t.reference, t.amount
FROM users u
LEFT JOIN transactions t ON u.user_id = t.sender_id
UNION
SELECT u.first_name, t.reference, t.amount
FROM users u
RIGHT JOIN transactions t ON u.user_id = t.sender_id;
```

JOIN Type	Returns	NULLs Appear When	Use When
-----------	---------	-------------------	----------

INNER JOIN	Only matching rows from both tables	Never	You only want rows that exist in both tables
LEFT JOIN	All rows from the left table, matched rows from the right	Right table has no match	You want every left-table row, even unmatched ones
RIGHT JOIN	All rows from the right table, matched rows from the left	Left table has no match	You want every right-table row, even unmatched ones
FULL JOIN	All rows from both tables	Either side has no match	You want all rows from both tables regardless of matches

5. How to Choose Which Table Goes Left or Right

One of the most important practical decisions when writing a JOIN is choosing which table to put in FROM (the left table) and which to JOIN to (the right table). This choice determines which rows are preserved in the result.

5.1 The Core Rule

The table you put in FROM is your LEFT table. Every row from the left table will appear in the result when you use a LEFT JOIN, even if there is no match in the right table.

Ask yourself: which table's rows must I never lose from the result? That table goes on the LEFT.

5.2 Step-by-Step Example: Students and Rooms

Scenario: LMU has a hostel. Some students have been assigned to a room. Others have not yet been assigned. You want to find the names of all students who have NOT been assigned to a room.

The two tables involved are:

- **students:** Contains all 50 students regardless of whether they have a room.
- **rooms:** Contains all available rooms. A room exists whether or not a student is in it.

Step through the decision process below:

Step 1: Identify what the question is about. The question asks about STUDENTS who have no room. The subject is students.

Step 2: Ask: which table must I never lose rows from? I need every student to appear in the result, even those with no room. That is the students table.

Step 3: Put students on the LEFT in FROM. The students table goes in FROM because its rows must all be preserved.

Step 4: Put rooms on the RIGHT in LEFT JOIN. Rooms is the optional side. If a student has no room, the rooms columns will just show NULL.

Step 5: Add the WHERE filter. Filter on room columns being NULL to find only the unassigned students.

```
-- Step 1-4: LEFT JOIN students to rooms
```

```

SELECT s.student_id, s.full_name, s.matric_num, r.room_number
FROM students s
LEFT JOIN rooms r ON s.room_id = r.room_id;

-- This returns ALL 50 students.
-- Students with a room show the room number.
-- Students without a room show NULL for room_number.

-- Step 5: Filter to only students with no room assigned
SELECT s.student_id, s.full_name, s.matric_num
FROM students s
LEFT JOIN rooms r ON s.room_id = r.room_id
WHERE r.room_id IS NULL;

```

The WHERE `r.room_id IS NULL` condition works because LEFT JOIN already returned NULL for the rooms columns of unmatched students. Filtering on NULL selects only those unmatched rows.

5.3 Why Not Put Rooms on the Left?

This is the key insight that many students miss. If you swap the tables and put rooms on the left instead of students, you change the question being asked entirely.

```

-- WRONG for this question: rooms on the left
SELECT s.full_name, r.room_number
FROM rooms r
LEFT JOIN students s ON r.room_id = s.room_id
WHERE s.student_id IS NULL;

-- This gives you rooms with NO student assigned to them.
-- It does NOT give you students with no room.
-- The subject changed from students to rooms.

```

Putting rooms on the left asks: which rooms have no student?
 Putting students on the left asks: which students have no room?

The table on the LEFT defines the subject of the question.

5.4 The 1:1 Relationship Case

In a true 1:1 relationship, every student has exactly one room and every room has exactly one student. But in practice, even in a 1:1 relationship, the assignment is rarely complete at all times. During the start of term, students arrive before rooms are assigned. This makes the relationship optionally 1:1 in practice.

Even in a strict 1:1 design, the LEFT vs RIGHT decision still matters because one side is typically the dependent side. The dependent entity is the one that cannot be meaningfully created without the other.

In the students and rooms example, a room exists independently of whether a student is in it. A student, however, is what is being assigned. The student record is the one that holds the `room_id` foreign key. Therefore the student is the dependent entity in the assignment relationship.

Step 1: Identify the dependent entity. Students hold the `room_id` foreign key. They are the dependent side.

Step 2: Identify what you are asking about. The question is about students without rooms.

Step 3: The entity you are asking about AND the dependent entity are the same: students. Students go on the LEFT.

Step 4: Rooms go on the RIGHT. The room will show NULL in the result if no student has that room_id.

Step 5: Add WHERE r.room_id IS NULL to filter only the unmatched students.

A simple way to remember it: the table that holds the foreign key column is often the one to put on the LEFT when you are looking for unmatched rows, because it is the table whose rows you want to audit.

5.5 Summary: The Decision Framework

Question to Ask	Answer	Decision
What is the subject of my question?	Students (or whichever table the question is about)	That table goes in FROM (LEFT)
Which table must I never lose rows from?	The table whose complete list I need	That table goes in FROM (LEFT)
Which table is optional in the relationship?	Rooms (not all students have one)	That table goes in JOIN (RIGHT)
What do I filter on to find unmatched rows?	The RIGHT table's PK or FK being IS NULL	Add WHERE right_table.id IS NULL

6. Multiple JOINS

A single query can join three or more tables. Each additional JOIN attaches a new table to the growing result set. The base table in FROM remains the anchor throughout, and each subsequent JOIN adds columns from the next table.

The same decision rule applies to each JOIN: if the new table is mandatory (the FK is NOT NULL), use INNER JOIN. If it is optional (the FK is nullable), use LEFT JOIN.

```
-- Three-table JOIN: transactions + sender (users) + agent (agents)
SELECT
    t.reference,
    t.transaction_type,
    t.amount,
    t.status,
    CONCAT(u.first_name, ' ', u.last_name) AS sender_name,
    u.city AS sender_city,
    a.agent_name,
    a.city AS agent_city
FROM transactions t
INNER JOIN users u ON t.sender_id = u.user_id -- sender always exists
LEFT JOIN agents a ON t.agent_id = a.agent_id -- agent is optional
WHERE t.transaction_type IN ('WITHDRAWAL', 'DEPOSIT')
ORDER BY t.amount DESC;
```

When the same table is joined more than once, you must give each join a different alias so MySQL can tell them apart. This is common when a table references the same parent table through multiple foreign keys, as in the transactions table which has both sender_id and receiver_id both pointing to users.

```
-- Joining users twice: once for sender, once for receiver
SELECT
    t.reference,
    t.amount,
    CONCAT(sender.first_name, ' ', sender.last_name) AS sender_name,
    CONCAT(receiver.first_name, ' ', receiver.last_name) AS
receiver_name,
    a.agent_name
FROM transactions t
INNER JOIN users sender ON t.sender_id = sender.user_id
LEFT JOIN users receiver ON t.receiver_id = receiver.user_id
LEFT JOIN agents a ON t.agent_id = a.agent_id;
```

Note: *receiver and sender are aliases for the same users table. Without different aliases, MySQL cannot distinguish between the two joins to the same table and will throw an error.*

7. GROUP BY, ORDER BY and HAVING

7.1 GROUP BY

GROUP BY groups rows that share the same value in specified columns into summary rows. It is always used with aggregate functions to perform a calculation on each group rather than on the full table.

```
-- Count how many employees are in each department
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees
FROM Employees
GROUP BY Department_ID;

-- Total transaction amount per transaction type
SELECT transaction_type, SUM(amount) AS total_amount
FROM transactions
GROUP BY transaction_type;
```

7.2 ORDER BY

ORDER BY sorts query results by one or more columns. ASC sorts from lowest to highest and is the default. DESC sorts from highest to lowest. Multiple columns can be specified with different sort directions for each.

```
-- Sort employees alphabetically by name
SELECT Name, Email
FROM Employees
ORDER BY Name ASC;

-- Departments sorted by number of employees, largest first
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees
FROM Employees
GROUP BY Department_ID
ORDER BY NumberOfEmployees DESC;
```

7.3 HAVING

HAVING filters the results of a GROUP BY. It works like WHERE but operates on aggregated groups rather than individual rows. WHERE cannot be used with aggregate functions. HAVING can.


```
-- Show only departments with more than 10 employees
SELECT Department_ID, COUNT(Employee_ID) AS NumberOfEmployees
FROM Employees
GROUP BY Department_ID
HAVING COUNT(Employee_ID) > 10;
```

Clause	Filters	Works With Aggregates?	Position in Query
WHERE	Individual rows before grouping	No	Before GROUP BY
HAVING	Groups after aggregation	Yes	After GROUP BY

7.4 Full Query Order

A complete SELECT query must follow this strict clause order. Clauses can be omitted if not needed, but the order of those used must never change.

```
SELECT    column_name(s), aggregate_function(column_name)
FROM      table_name
JOIN      other_table ON condition
WHERE     row-level condition
GROUP BY  column_name(s)
HAVING    group-level condition
ORDER BY  column_name(s) [ASC | DESC]
LIMIT    number;
```

8. Subqueries

A subquery is a query nested inside another query. It is used when the value or set of values needed to filter or calculate results cannot be determined without first running another query. The inner query (subquery) runs first and its result is used by the outer query.

Subqueries can appear in three places within a main query:

- **WHERE clause:** To filter rows based on a value that must be calculated dynamically.
- **FROM clause:** To define a temporary inline table that the outer query treats like a regular table. This is sometimes called a derived table.
- **SELECT clause:** To calculate a value for each row returned by the outer query.
- **HAVING clause:** To filter groups using a dynamically calculated aggregate value.

8.1 Types of Subqueries

Type	Returns	Example Use
Scalar subquery	A single value	WHERE age > (SELECT AVG(age) FROM employees)
Column subquery	A single column of values	WHERE city IN (SELECT city FROM active_branches)
Table subquery	Multiple rows and columns	FROM (SELECT ...) AS derived_table
Correlated	Value depends on outer query	Each row of outer query triggers the inner query

Uncorrelated	Independent of outer query	Inner query runs once, result reused
--------------	----------------------------	--------------------------------------

8.2 Subquery in WHERE

This is the most common placement. The subquery calculates a value and the outer query uses it as a filter condition.

```
-- Find employees older than the average age
SELECT Name, Age
FROM Employees
WHERE Age > (SELECT AVG(Age) FROM Employees);

-- Find users with a balance above the average balance
SELECT first_name, last_name, balance
FROM users
WHERE balance > (SELECT AVG(balance) FROM users);
```

8.3 Subquery in FROM

The subquery acts as a derived table. It must be given an alias so the outer query can reference it.

```
-- Use the result of a subquery as a table
SELECT avg_data.AverageAge
FROM (SELECT AVG(Age) AS AverageAge FROM Employees) AS avg_data;

-- Find the top-spending city using a derived table
SELECT city_totals.city, city_totals.total
FROM (
    SELECT u.city, SUM(t.amount) AS total
    FROM transactions t
    INNER JOIN users u ON t.sender_id = u.user_id
    GROUP BY u.city
) AS city_totals
ORDER BY city_totals.total DESC
LIMIT 1;
```

8.4 Subquery in HAVING

When a HAVING condition requires a dynamically calculated aggregate value from the same or another table, a subquery provides that value.

```
-- Show departments whose average salary is above the company-wide average
SELECT DepartmentID, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY DepartmentID
HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employees);
```

8.5 Correlated vs Uncorrelated Subqueries

An uncorrelated subquery runs independently of the outer query. It executes once and its result is reused for every row of the outer query. This is efficient.

A correlated subquery references a column from the outer query. It must be re-executed for every single row of the outer query. This can be slow on large tables but is sometimes necessary.

```
-- Uncorrelated: inner query runs ONCE
SELECT Name FROM Employees
WHERE Age > (SELECT AVG(Age) FROM Employees);

-- Correlated: inner query runs ONCE PER ROW of outer query
SELECT e1.Name, e1.Salary
FROM Employees e1
WHERE e1.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentID = e1.DepartmentID -- references outer query
);
-- This finds employees who earn above their own department's average
```

9. Views

A view is a virtual table based on the result of a stored SELECT query. It contains rows and columns like a real table, but does not store any data itself. Every time the view is queried, MySQL re-executes the underlying SELECT against the base tables and returns the current result.

9.1 Uses of Views

- **Simplify complex queries:** A complex JOIN or aggregation can be saved as a view. Users then query the view with a simple SELECT instead of rewriting the full query every time.
- **Restrict data access:** A view can expose only certain columns or rows to a user, hiding sensitive information such as salaries or personal details without changing the underlying table.
- **Present computed data:** Aggregations, calculated columns, and formatted results can be presented through a view as if they were stored values.

9.2 Creating and Using Views

```
-- Create a view
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;

-- Query the view like a normal table
SELECT * FROM view_name;

-- Real example: view of successful transactions only
CREATE VIEW v_successful_transactions AS
SELECT * FROM transactions WHERE status = 'SUCCESS';

SELECT * FROM v_successful_transactions ORDER BY amount DESC LIMIT 5;

-- Drop a view
DROP VIEW IF EXISTS v_successful_transactions;
```

9.3 Views vs Temporary Tables

	Views	Temporary Tables
Stores data?	No, only stores the query definition	Yes, stores actual rows

Persists?	Yes, survives after session ends	No, dropped when session closes
Reflects changes?	Yes, always shows current data	No, snapshot at creation time
Performance	Recalculated each access; slower on large datasets	Faster for repeated access since data is stored once
Memory	Minimal, only stores query text	Higher, stores all data rows
Scope	Accessible to any user with permission	Only visible within the current session
Best for	Simplifying queries, restricting access	Intermediate results, complex multi-step calculations

10. Stored Procedures

A stored procedure is a set of pre-written SQL statements saved in the database under a name. It can be called by that name whenever needed, avoiding the need to rewrite the same SQL code repeatedly. Stored procedures are supported in MySQL, PostgreSQL, SQL Server, and Oracle.

10.1 Key Features

- **Modularity:** SQL code is encapsulated into named, reusable modules. Complex logic is broken into specific tasks that are easier to maintain and understand.
- **Performance:** Once created, a stored procedure is compiled and stored. The database can reuse the execution plan without parsing and compiling the SQL code on each call.
- **Reduced Network Traffic:** Only the CALL statement is sent over the network rather than multiple lines of SQL, reducing bandwidth between client and server.
- **Security:** Users can be granted access to execute a stored procedure without being given direct access to the underlying tables. The procedure controls exactly what data is touched.
- **Reusability and Maintainability:** A procedure can be called from multiple applications. When the logic changes, only the procedure needs updating, not every application that uses it.

10.2 Syntax

The DELIMITER keyword is used to change the statement terminator from the default semicolon to something else, typically \$\$\$. This is necessary because the procedure body itself contains semicolons at the end of each statement. Without changing the delimiter, MySQL would incorrectly interpret those semicolons as the end of the CREATE PROCEDURE command itself.

```
DELIMITER $$
CREATE PROCEDURE procedure_name (parameter_list)
BEGIN
    -- SQL statements go here
END$$
DELIMITER ;
```

10.3 Parameter Types

Parameter Type	Direction	Description
IN	Caller to procedure	A value passed into the procedure. The procedure can read it but cannot modify the caller's original variable.

OUT	Procedure to caller	A variable the procedure sets. The caller reads its value after the procedure finishes.
INOUT	Both directions	A value passed in that the procedure can modify, with the modified value passed back to the caller.

10.4 Example

```

DELIMITER $$
CREATE PROCEDURE GetEmployeesByDepartment(IN DeptID INT)
BEGIN
    SELECT * FROM Employees WHERE DepartmentID = DeptID;
END$$
DELIMITER;

-- Call the stored procedure
CALL GetEmployeesByDepartment(3);

-- PocketPay example: get all transactions for a given user
DELIMITER $$
CREATE PROCEDURE GetUserTransactions(IN p_user_id INT)
BEGIN
    SELECT t.reference, t.transaction_type, t.amount, t.status
    FROM transactions t
    WHERE t.sender_id = p_user_id
    ORDER BY t.transaction_date DESC;
END$$
DELIMITER ;

CALL GetUserTransactions(5);

```

Summary

In this module, we covered the full picture of data manipulation and retrieval in SQL. DML commands (INSERT, SELECT, UPDATE, DELETE) are used to manage data, and WHERE must always be used with UPDATE and DELETE to avoid affecting the entire table.

JOINS are the mechanism for combining data from related tables. INNER JOIN returns only matched rows. LEFT JOIN keeps all rows from the left table. RIGHT JOIN keeps all rows from the right table. The choice of which table goes left is determined by asking which table's rows must never be lost from the result. In the students and rooms example, students goes left because the question is about students, and the filter on NULL room columns then isolates those without an assignment.

GROUP BY groups rows for aggregation. ORDER BY sorts the result. HAVING filters aggregated groups after GROUP BY has run, while WHERE filters individual rows before grouping. The clause order in a complete query is: SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT.

Subqueries nest one query inside another. They can appear in WHERE to filter on dynamic values, in FROM to create derived tables, in SELECT to calculate per-row values, or in HAVING to filter groups using a dynamic aggregate. Correlated subqueries depend on the outer query and re-execute per row; uncorrelated subqueries run once independently.

Views are virtual tables that store a SELECT query, not data. They always reflect current data, persist after sessions end, and are used to simplify queries and restrict access. Temporary tables store actual data, only exist for the session, and are better for intermediate results in complex calculations. Stored

procedures are reusable, named SQL programs that improve performance, security, and maintainability by encapsulating database logic.